

LLM APIs – How to Access & Use Hosted Language Models

✦ Based on Course Video: *Accessing Hosted Models (Groq Cloud Demo)*

Goal of this note: Understand what hosted LLM APIs are, how they work end-to-end, and what production-grade usage looks like — from signing up to making your first API call.

Table of Contents

1. [What is a Hosted LLM API?](#)
 2. [Groq Cloud – Why It's Used Here](#)
 3. [Plans, Rate Limits & Production Implications](#)
 4. [The Playground – Your Experimentation Sandbox](#)
 5. [System Prompts – The First Layer of Prompt Engineering](#)
 6. [API Keys – Authentication & Security](#)
 7. [Making Your First API Call \(curl\)](#)
 8. [The JSON Response – Anatomy of an LLM Response](#)
 9. [Groq vs Other LLM Providers](#)
 10. [Production Checklist](#)
 11. [Key Takeaways](#)
-

1. What is a Hosted LLM API?

A **Hosted LLM API** means the AI model is running on someone else's servers (the provider's cloud), and you access it over the internet via HTTP requests.

Your Code / Terminal

|

| HTTP Request (with your prompt)

▼

LLM Provider's Servers

(Model runs here – Groq, OpenAI, Anthropic, etc.)

|

| HTTP Response (with model's reply as JSON)

▼

Your Code / Terminal

You do NOT need to:

- Download or install the model
- Have a GPU
- Manage model infrastructure

You DO need to:

- An account with the provider
- An API key (your authentication credential)
- Send HTTP requests in the right format

Why this matters in production:

In enterprise systems, you almost never run models locally. You call hosted APIs because they are scalable, managed, and updated by the provider. Your job as a developer is to **integrate with** the model, not **run** it.

2. Groq Cloud - Why It's Used Here

Groq is an LLM API provider, but with a twist — they use custom hardware called **LPU (Language Processing Unit)** specifically designed for fast inference (generating tokens quickly).

Feature	Groq
Speed	Extremely fast (often 300-500 tokens/sec vs OpenAI's ~60)
Models Available	Llama 3, Mixtral, Gemma, and others (open-source models)
Free Plan	Yes — with rate limits
Own Models?	No — they host open-source models on their hardware

Key insight:

Groq does **not make its own models**. It takes existing open-source models (like Meta's Llama) and runs them on its fast hardware. You get the speed of Groq's infrastructure with the intelligence of the open-source models.

3. Plans, Rate Limits & Production Implications

When you sign up for Groq (or any LLM provider), you choose a **plan** that determines how much you can use the API.

Free Plan Limits (Groq example):

Limit Type	Value
Requests per minute (RPM)	30
Requests per day	Capped per model
Tokens per minute (TPM)	Capped
Tokens per day	Capped

What is a Token?

A token is roughly $\frac{3}{4}$ of a word. The sentence *"How does a car work?"* is approximately 7 tokens.

"How does a car work?" → ["How", " does", " a", " car", " work", "?"] ≈ 6-7 tokens

Both your **input** (the prompt you send) and the **output** (the model's reply) consume tokens.

Production Implication:

The free plan is **only for learning and prototyping**. In production:

- A real app might make hundreds or thousands of requests per minute
- You need the **paid plan** (Developer or Business tier)
- You must implement **rate limit error handling** — if you exceed limits, the API returns an error (HTTP 429), and your app must retry gracefully

Rate Limit Handling Pattern (for later implementation):

Request → API
If 429 error → Wait → Retry (with exponential backoff)
If success → Process response

4. The Playground - Your Experimentation Sandbox

The **Playground** is a browser-based UI that lets you interact with models without writing a single line of code.

What you can do in the Playground:

- Type a **system prompt** (defines how the model behaves)
- Type a **user message** (your actual question)
- Select different **models** from a dropdown
- Adjust **inference parameters** (temperature, max tokens — covered in later lessons)
- See the model's response instantly

When to use the Playground:

Situation	Use Playground?
Testing a new system prompt	✓ Yes
Comparing model responses	✓ Yes
Building a production feature	✗ No — use code
Debugging why a model responds poorly	✓ Yes

Production insight:

⌋ Treat the Playground as your **prompt design lab**. Before hardcoding any prompt into your application, always test and refine it in the Playground first. This saves debugging time later.

5. System Prompts - The First Layer of Prompt Engineering

In the demo, the instructor pastes this system prompt:

⌋ *"You are a smart science teacher for grade five. Politely answer the question."*

What is a System Prompt?

A system prompt is a **set of instructions given to the model before the user's message**. It defines:

- The model's **role/persona** (e.g., "You are a science teacher")

- Its **behavior** (e.g., "Be polite", "Be concise")
- Its **constraints** (e.g., "Only answer science questions", "Never mention competitor products")
- Its **output format** (e.g., "Always respond in bullet points")

How it fits into a conversation:

```
SYSTEM PROMPT
```

```
"You are a science teacher for grade 5..."
```

```
USER MESSAGE
```

```
"How does a car work?"
```

```
ASSISTANT RESPONSE
```

```
"Great question! A car works by..."
```

Production insight:

In real applications, system prompts are:

- **Version controlled** (stored in Git, not hardcoded randomly)
- **Tested like code** (if you change the system prompt, you re-test all expected behaviors)
- **The single most impactful thing** you control about how your AI feature behaves
- Often the "**secret sauce**" of a product — they are treated as intellectual property

6. API Keys - Authentication & Security

An **API key** is a unique secret string that proves to the LLM provider that *you* are making the request. It's like a password for your application.

How the instructor handled it (correct way):

```
bash
```

```
# Set the API key as an environment variable - NOT hardcoded  
export GROQ_API_KEY="your_actual_key_here"
```

Then in the curl command:

```
bash
```

```
-H "Authorization: Bearer $GROQ_API_KEY"
```

The 3 Rules of API Key Security:

Rule 1: Never hardcode it in your source code

```
python
```

```
# ❌ WRONG – Never do this
```

```
client = Groq(api_key="gsk_abc123xyz...")
```

```
# ✅ CORRECT – Read from environment
```

```
import os
```

```
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
```

Rule 2: Never commit it to GitHub

```
# Your .gitignore must include:
```

```
.env
```

```
*.env
```

```
secrets.txt
```

Rule 3: Rotate it if exposed

If your API key ever accidentally gets pushed to a public GitHub repo, **immediately go to the provider's dashboard and delete it, then create a new one**. Bots scan GitHub continuously for exposed API keys.

Where API keys live in different environments:

Environment	Where to Store Key
Local development	<code>.env</code> file (not committed to git)
CI/CD pipelines	GitHub Secrets / GitLab CI Variables
Production servers	AWS Secrets Manager / Azure Key Vault / HashiCorp Vault

7. Making Your First API Call (curl)

The instructor used a `curl` command from the terminal to call the model. This is a raw HTTP request.

What curl is:

`curl` is a command-line tool to make HTTP requests. When you call an LLM API from Python later, it's doing the **exact same thing under the hood** — just wrapped in nicer syntax.

The curl command pattern:

```
bash
curl https://api.groq.com/openai/v1/chat/completions \
  -H "Authorization: Bearer $GROQ_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "model": "llama3-8b-8192",
    "messages": [
      {
        "role": "system",
        "content": "You are a helpful assistant."
      },
      {
        "role": "user",
        "content": "How does a car work?"
      }
    ]
  }'
```

Breaking it down:

Part	What it does
<code>https://api.groq.com/openai/v1/chat/completions</code>	The API endpoint URL
<code>-H "Authorization: Bearer \$GROQ_API_KEY"</code>	Authenticates your request using your API key
<code>-H "Content-Type: application/json"</code>	Tells the server you're sending JSON
<code>"model": "llama3-8b-8192"</code>	Which model to use

Part	What it does
<code>"messages": [...]</code>	The conversation (system + user messages)

Why "openai/v1" in Groq's URL?

Groq's API is **OpenAI-compatible**. This means the same Python code you write for OpenAI works with Groq by just changing the base URL and API key. This is a deliberate design decision by Groq to make it easy to switch.

8. The JSON Response - Anatomy of an LLM Response

When you call the API, you get back a **JSON object**. Here's what a typical response looks like:

```
json
{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
  "created": 1710000000,
  "model": "llama3-8b-8192",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "A car works by converting fuel into motion through an internal c
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 28,
    "completion_tokens": 142,
    "total_tokens": 170
  }
}
```


Key fields explained:

Field	Meaning
<code>choices[0].message.content</code>	The actual text response from the model
<code>choices[0].finish_reason</code>	Why the model stopped (<code>stop</code> = natural end, <code>length</code> = hit token limit)
<code>usage.prompt_tokens</code>	Tokens consumed by your input
<code>usage.completion_tokens</code>	Tokens consumed by the model's output
<code>usage.total_tokens</code>	Total tokens (this is what you get billed for)

Production insight:

Always log `usage.total_tokens` in production. This is how you track costs and detect runaway prompts that consume too many tokens. $\text{Cost} = \text{total_tokens} \times \text{price_per_token}$ (varies by model).

9. Groq vs Other LLM Providers

The API pattern you just learned is nearly identical across all major providers. Here's a comparison:

Provider	Models	Free Tier	API Style	Best For
Groq	Llama 3, Mixtral, Gemma	✔ Yes	OpenAI-compatible	Speed, prototyping
OpenAI	GPT-4o, GPT-4, GPT-3.5	✗ No (paid only)	OpenAI (standard)	General purpose, most capable
Anthropic	Claude 3.5 Sonnet, Haiku	✗ No	Similar to OpenAI	Long context, safety, reasoning
Google	Gemini 1.5 Pro/Flash	✔ Yes (limited)	Google SDK	Multimodal, large context
Ollama	Any open-source model	✔ Free (local)	OpenAI-compatible	Offline, privacy-sensitive apps

Key insight:

Learning one API means you can use all of them. The concepts — API key, system prompt, messages array, JSON response, token usage — are **universal**.

10. Production Checklist

Before shipping any feature that calls an LLM API:

- ☐ API key stored in environment variable, NOT in code
 - ☐ `.env` added to `.gitignore`
 - ☐ Rate limit errors handled (retry with backoff on HTTP 429)
 - ☐ Token usage logged for cost tracking
 - ☐ System prompt version controlled in Git
 - ☐ Model name configurable (not hardcoded) — so you can swap models easily
 - ☐ Timeout set on API calls — LLM calls can be slow, don't let them hang forever
 - ☐ Response validated before using — model can return unexpected formats
-

11. Key Takeaways

- A **hosted LLM API** lets you use powerful AI models without managing any infrastructure
 - **Groq** is fast because of custom LPU hardware; it hosts open-source models
 - The **Playground** is for experimentation; actual applications use code/API calls
 - **System prompts** define model behavior — they are the most important thing you control
 - **API keys** must be stored securely — never in code, never committed to Git
 - Every API call returns a **JSON response** — the actual text is in `choices[0].message.content`
 - **Token usage** in the response tells you cost — always track it in production
 - This same pattern (API key + messages array + JSON response) works across **all LLM providers**
-

What Comes Next

This lesson sets the foundation for everything that follows:

LLM API Basics (this lesson)



Calling APIs from Python (SDK usage)



Prompt Engineering (system prompts, few-shot, chain-of-thought)



Building Chains (connecting multiple LLM calls)



RAG (grounding the model with your own data)



Agentic Systems (LLMs that take actions)

Notes by Siva Sai Charan / Gen AI Application Development Course